Einführung in die Rechnerarchitektur (ERA)
IN0004
Einführung Assembler

Martin Schulz
schulzm@in.tum.de
Chair for Computer Architecture and Parallel Systems
https://www.ce.cit.tum.de/caps/

Robert Wille
robert.wille@tum.de
Chair for Design Automation
https://www.cda.cit.tum.de/



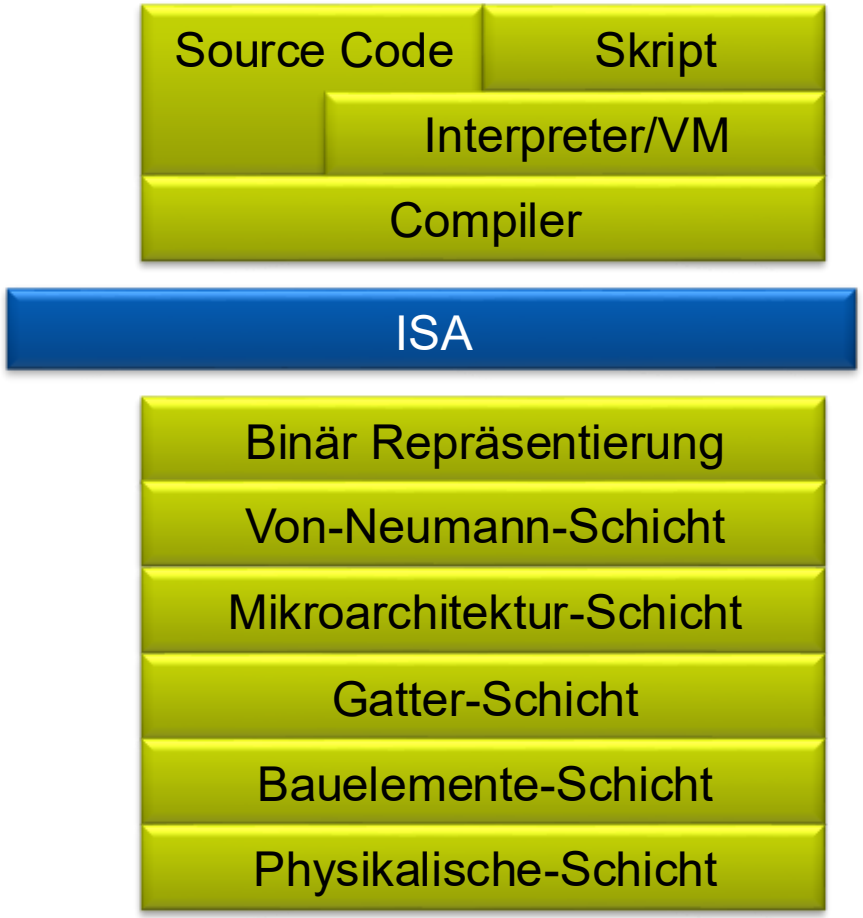
Wiederholung Abstraktionsschichten

Inhalte Rechnerarchitektur
- Umsetzung/Ausführung von Code
- Systemarchitektur
  - V.a. Speichersysteme
- Design von Prozessoren
- Umsetzung in einzelne Schaltungen

Stark geschichtete System
- Abstraktionsebenen
- ISA sichtbar für Programmierer
- Verständnis wichtig für Performanz

Abstraktionen erlauben den Austausch von Ebenen
- Ohne Programmierung zu verändern
- Passende für verschiedene Optimierungsziele

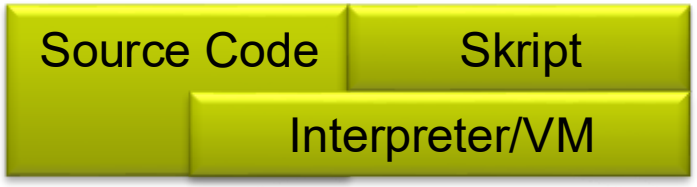
Vieles sind Designentscheidungen!



Programm-Abstraktion

Eingabe: Code

- Kodiert als Text
- Nicht direkt ausführbar



C Code

```
int main(int argc, char *argv[])
{
    int c=0;
    for (int i=0; i<10; i++)
      { c = c + i;}
    printf("Ergebnis ist %i\n",c);
}
```

Programm-Abstraktion

Eingabe: Code

- Kodiert als Text
- Nicht direkt ausführbar

C Code

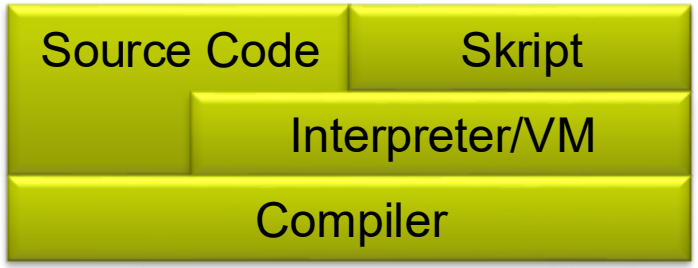
```
int main(int argc, char *argv[])
{
    int c=0;
    for (int i=0; i<10; i++)
      { c = c + i;}
    printf("Ergebnis ist %i\n",c);
}
```

- Übersetzen in ausführbare Datei (Linux, z.B. Ubuntu)

```
>> gcc -o count -O0 -g count.c
```

- Ausführung des Programmes

```
>> ./count
Ergebnis ist 45
>>
```



Compiler Optionen

-o = Name der Ausgabedatei
-O<n> = Optimierungsgrad (0,1,2,…)
-g = Extra Symbole

Für weitere, siehe: man gcc

Programm-Abstraktion

Eingabe: Code

- Kodiert als Text
- Nicht direkt ausführbar

C Code

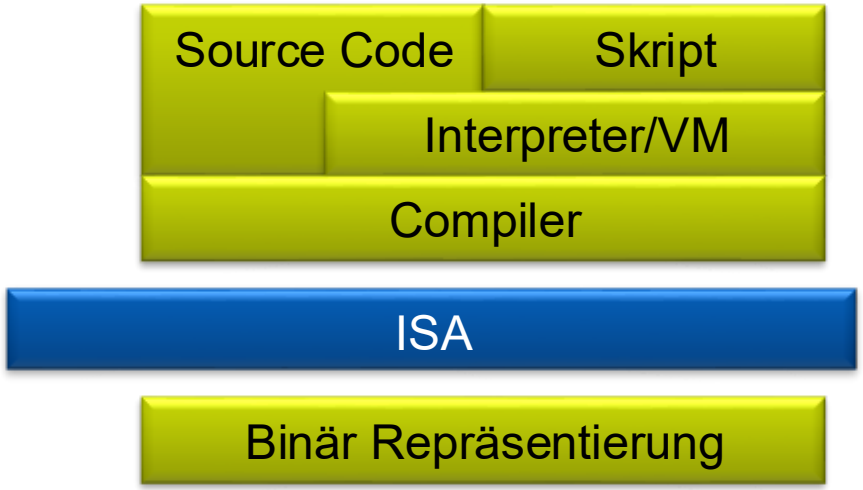
```
int main(int argc, char *argv[])
{
    int c=0;
    for (int i=0; i<10; i++)
      { c = c + i;}
    printf("Ergebnis ist %i\n",c);
}
```

- Übersetzen in ausführbare Datei (Linux, z.B. Ubuntu)

```
>> gcc -o count -O0 -g count.c
```

- Ausführung des Programmes

```
>> ./count
Ergebnis ist 45
>>
```





Binär-Code

Ausführbare Dateien

Binär Dateien

- Reihen von 0/1, nicht (ohne weiteres) lesbar

```
>> hexdump count
0000000 457f 464c 0102 0001 0000 0000 0000 0000
0000010 0003 00f3 0001 0000 05b0 0000 0000 0000
0000020 0040 0000 0000 0000 1a70 0000 0000 0000
0000030 0005 0000 0040 0038 000a 0040 001c 001b
…
```

Was ist darin kodiert?

- Programm
- Startwerte für Daten
- Metadaten

Format und Inhalte hängen ab von

- Betriebssystem
- Hardware Plattform

Was ist in einer ausführbaren Binärdatei

Auslesen mit `objdump`

- Alle „Headers“ (Dateiteile) mit „-h“

```
>> objdump -h count

count:     file format elf64-littleriscv

Sections:
Idx Name          Size      VMA               LMA               File off  Algn
  0 .interp       00000021  0000000000000270  0000000000000270  00000270  2**0
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  1 .note.gnu.build-id 00000024  0000000000000294  0000000000000294  00000294  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  2 .note.ABI-tag 00000020  00000000000002b8  00000000000002b8  000002b8  2**2

…
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
 11 .text         0000011a  00000000000005b0  00000000000005b0  000005b0  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, CODE

…
```

Was ist in einer ausführbaren Binärdatei

Auslesen mit `objdump`

- Alle „Headers“ (Dateiteile) mit „-h“
- Codiertes Programm mit „-d“ = Dissassemblierung

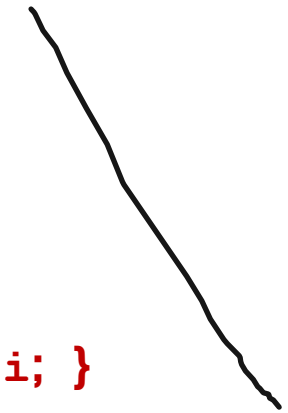
```
0000000000000668 <main>:
 668: 7179            add sp,sp,-48
 66a: f406            sd ra,40(sp)
 66c: f022            sd s0,32(sp)
 66e: 1800            add s0,sp,48
 670: 87aa            mv a5,a0
 672: fcb43823        sd a1,-48(s0)
 676: fcf42e23        sw a5,-36(s0)
 67a: fe042423        sw zero,-24(s0)
 67e: fe042623        sw zero,-20(s0)
 682: a831            j 69e <main+0x36>
 684: fe842783        lw a5,-24(s0)
 688: 873e            mv a4,a5
 68a: fec42783        lw a5,-20(s0)
 68e: 9fb9            addw a5,a5,a4
 …
```

Was ist in einer ausführbaren Binärdatei

Auslesen mit `objdump`

- Alle „Headers“ (Dateiteile) mit „-h“
- Codiertes Programm mit „-d“ = Dissassemblierung
- Codiertes Programm plus Bezug auf Quellcode mit „-S“

```
    {
      c = c + i;
 684: fe842783        lw a5,-24(s0)
 688: 873e            mv a4,a5
 68a: fec42783        lw a5,-20(s0)
 68e: 9fb9            addw a5,a5,a4
 690: fef42423        sw a5,-24(s0)
    for (int i=0; i<10; i++) { c = c + i; }
 694: fec42783        lw a5,-20(s0)
 698: 2785            addw a5,a5,1
 69a: fef42623        sw a5,-20(s0)
 69e: fec42783        lw a5,-20(s0)
 6a2: 0007871b        sext.w a4,a5
 6a6: 47a5            li a5,9
 6a8: fce7dee3        bge a5,a4,684 <main+0x1c>
    }
```



Was ist in einer ausführbaren Binärdatei

Auslesen mit `objdump`

- Alle „Headers“ (Dateiteile) mit „-h“
- Codiertes Programm mit „-d“ = Disassemblierung
- Codiertes Programm plus Bezug auf Quellcode mit „-S“

```
    c = c + i;
684: fe842783        lw a5,-24(s0)
688: 873e            mv a4,a5
68a: fec42783        lw a5,-20(s0)
68e: 9fb9            addw a5,a5,a4
690: fef42423        sw a5,-24(s0)
```

Fortlaufende Adressen

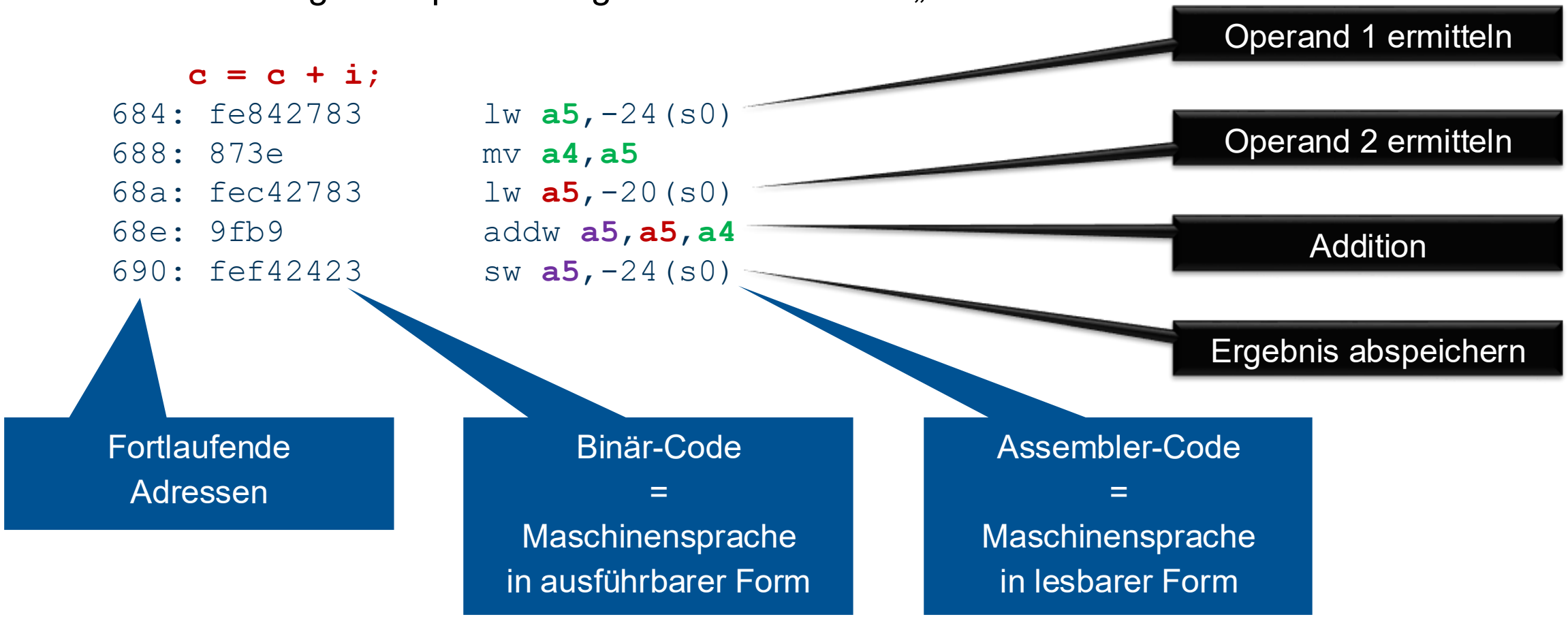
Binär-Code = Maschinensprache in ausführbarer Form

Assembler-Code = Maschinensprache in lesbarer Form

Was ist in einer ausführbaren Binärdatei

Auslesen mit `objdump`

- Alle „Headers“ (Dateiteile) mit „-h“
- Codiertes Programm mit „-d“ = Dissassemblierung
- Codiertes Programm plus Bezug auf Quellcode mit „-S“



Assemblersprache

Maschinen-spezifische Programmiersprache

- Niedrigste Abstraktionsbene für NutzerInnen
- Primitive Befehle
  - Elementare Berechnungsschritte
  - Keine komplexen Datenstrukturen
- 1:1 Übersetzung in Binärcode
- Manuelle Verwaltung von Resourcen
  - Speicherelemente
  - Datenübertragungen

Oft nahe and der Implementierung

- Einzelne Befehle direkt implementiert
- Kodierung passend zur internen Architektur
- Festgelegte “Breite”
  - Daten und/oder Instruktionen
- Limitierungen möglich
- Oft spezifische Erweiterungen

Drei verschiedene Gesichtspunkte

- Compiler → Binärdatei → Disassemblierung
  - Nur minimale extra Information

- Compiler → Assembler-Datei
  - Assemblercode ist Zwischenschritt (sonst meist versteckt)
  - `gcc -O0 count.c -g -s`
  - Ohne Adressen, mit generierten Labels
  - Wird anschließend assembliert

- Direkte Programmierung
  - Eigene Programmiersprache
  - Ohne Adressen
  - Hilfsmittel wie Labels, Kommentare, …
  - Wird anschließend assembliert
  - Meist nur für Kernroutinen

Assemblersprache sind Plattform spezifisch

Oft gruppiert in Familien

- Intel und AMD's IA32 und x86_64
  - Gewachsen von 8bit bis 64bit
  - Intel vs. AMD: fast gleiche Sprache, aber komplett unabhängige Implementierung
  - Dominant in PCs
- ARM
  - Keine eigene Herstellung
  - Lizenzierung der Sprache
  - Von embedded bis HPC
- IBM's Power Architektur
- Viele "ehemalige" Ansätze wie MIPS, Sparc
- Spezial-Ansätze, z.B. Atmel AVR

- **Offener Ansatz: RISC-V**

Beispiel: x86 Familie

- 8086 (16 bit) ➔ 80286 ➔ … Pentium (32 bit)
  ➔ … Pentium4/Netburst (64 bit)
  ➔ … „Core"-Architektur
- Neue Wortbreiten durch neue Ausführungsmodi
- Alle alten Ausführungsmodi weiterhin unterstützt
- ABER: jede Wortbreite definiert eigene ISA
- Lizensiert an AMD in den 80ern

IA-32: 32bit Variante

- Intel 80386, 80486, Intel Pentium P5 und P6
- Moderne ISA, but limitierte Komplexität
- Typische CISC ISA (sehr komplex)

Später: Erweiterung auf 64bit (x86_64)

- Erste x86 64-Bit-Erweiterung von AMD
- „Intel 64" und „AMD64" fast identisch

RISC-V

- Frei und offen
  - Jeder kann RISC-V ISA nutzen
  - Keine Lizenzkosten
  - Mehr Wettbewerb
- Einfach und modular
  - 5. RISC Architektur von UC Berkeley
  - Viel einfacher als ARM und x86
  - Erweiterbar: Custom-Instruktionen
- Für Cloud und Edge
  - Von Mikrocontrollern bis zu Supercomputern
- Security
  - Entwurf von eigenen Cores
  - Offene Cores
    - ➔ keine Geheimnisse
- Community Unterstützung und Stabilität
  - RISC-V Foundation „besitzt“ RISC-V Spezifikation
    - https://riscv.org
    - https://riscv.org/specifications/

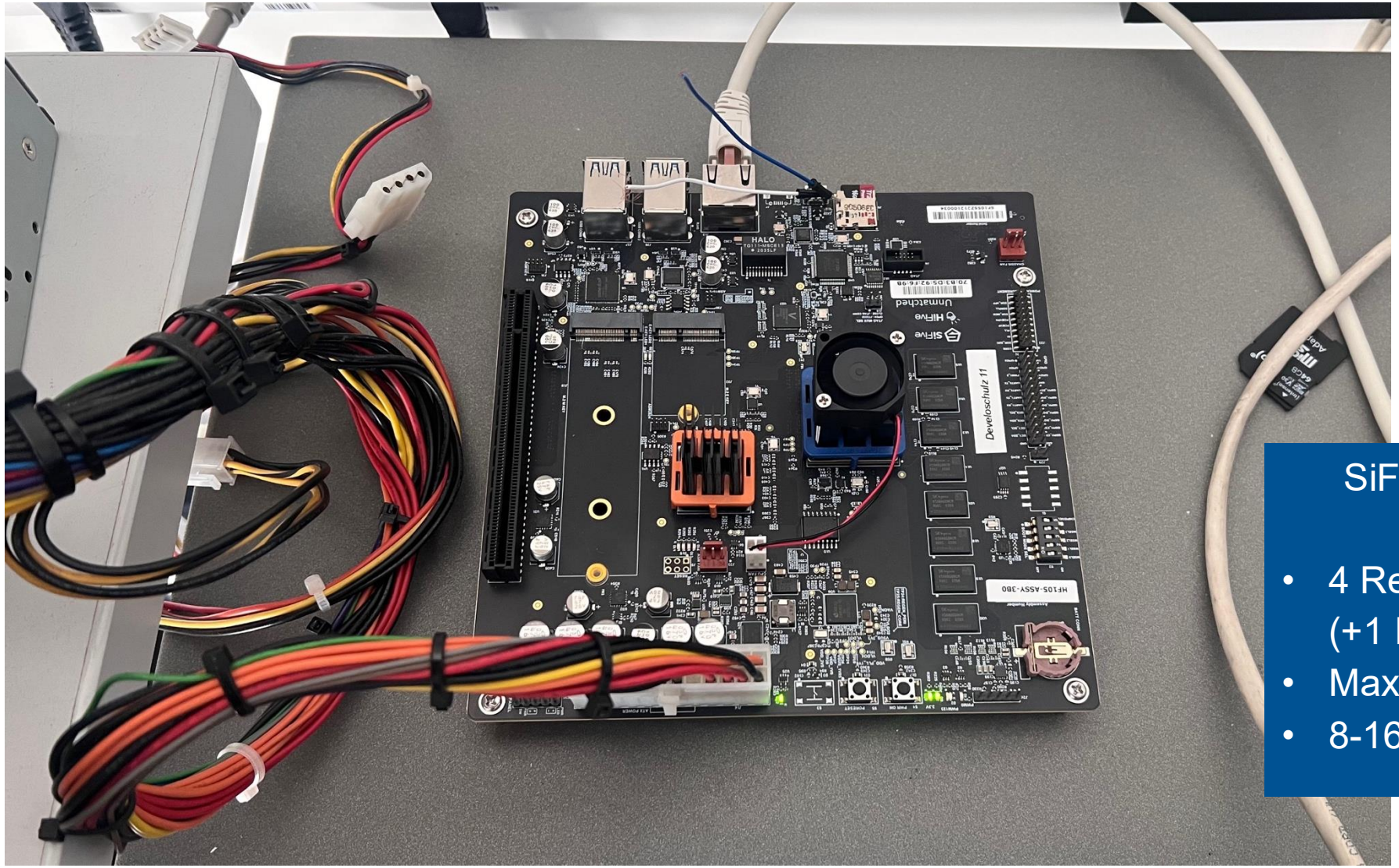




RISC-V Foundation >580 Mitglieder



RISC-V Plattform



SiFive HiFive Unmatched

- 4 Rechenkerne (64bit) (+1 RT Kern)
- Max 1.2 GHz
- 8-16 GB RAM

Mehr Information: https://www.tomshardware.com/news/sifive-launches-hifive-unmatched-risc-v-development-board-for-desktops
https://www.hardwareluxx.de/index.php/artikel/hardware/prozessoren/57184-ein-blick-in-die-zukunft-sifive-hifive-unmatched-angeschaut.html

RISC-V Varianten

Unterstützung von 3 verschiedenen Datenbreiten
- 32, 64 und 128 bit

Jeweils einfacher Satz von Basisinstruktionen
- Fokus: Berechnung mit ganzen Zahlen (Integers)

Erweiterungen
- Fliesskommazahlen
  - Verschiedene Optionen
- Vektoroperationen
- Mehrbenutzerbetrieb
- Multi-Core

Spezial-Erweiterungen möglich

https://riscv.org/technical/specifications/

```
>> cat /proc/cpuinfo
processor : 0
hart : 1
isa : rv64imafdc
mmu : sv39
uarch : sifive,u74-mc
mvendorid : 0x489
marchid : 0x8000000000000007
mimpid : 0x20181004
```

Standard-Namenskonvention für RISC-V
- RV[<Breite>][abc…xyz]

Diese Plattform: rv64imafdc (64bit)
- i = Integer (Basisinstruktionen)
- m = integer mult & div
- a = atomics
- f = 32bit Gleitkommazahlen
- d = 64bit Gleitkommazahlen
- c = Compressed Instructions

ISAs = Instruction Set Architectures

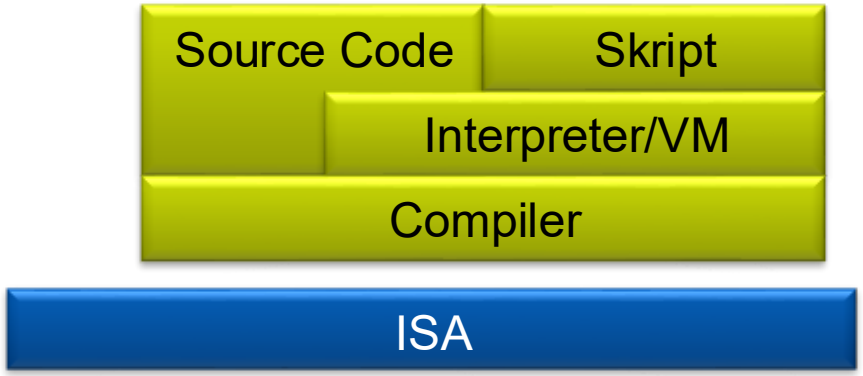
Assembler-Sprache
- Daten-Breite
- Vorhandene Befehle und deren Kodierung
  - Verschiedene Operationen
  - Verschiedene Varianten
- Erweiterungen

Unterstützte Datenkodierungen

Systemeigenschaften
- Speicher
- Power/Energy Kontrolle

Boot-Up Prozess
- Konfigurationen und Konfigurationsabfrage

Gesamter sichtbarer Zustand



Einzige Programmiersprache, die ein Prozessor verstehen kann

=

Zentrale Nutzer-Schnittstelle

Komplexität der ISA: RISC vs. CISC

CISC: Complex Instruction Set Computer

- Komfortabel und relativ mächtig
- Vorteil: einfache Programmierbarkeit,
- Realisiert durch Mikroprogramme
- Nachteile: komplexe (langsame) Implementierung insbesondere Dekodierung
  - Evtl. viele Funktionen ungenutzt

RISC: Reduced Instruction Set Computer

- Minimalistisch, auf das absolut Notwendige beschränkt
- Vorteil: einfache, effiziente, schnelle Implementierung
- Realisiert durch feste Verdrahtung
- Nachteil: schwierigere Programmierbarkeit
  - Aber: das erledigt der Compiler (!?)

Heute: Oft keine saubere Trennung mehr möglich

- CISC Architekturen haben einen RISC Kern
- Erweiterungen in RISC Architekturen oft CISC-like

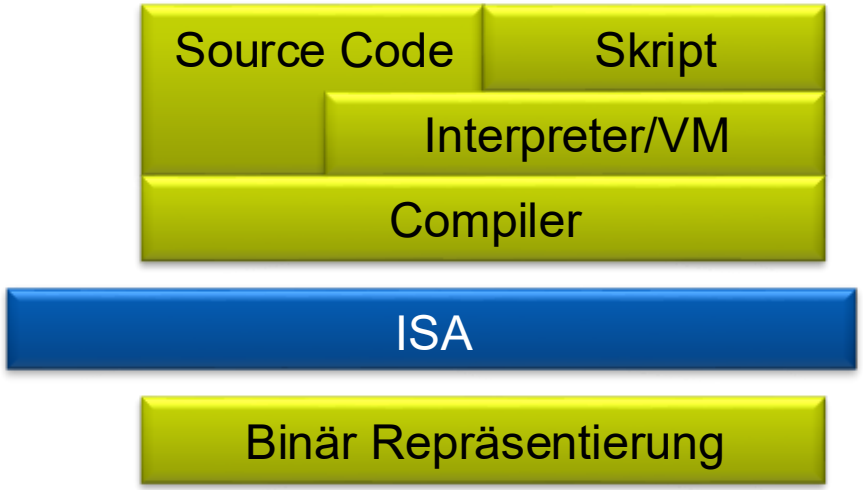
Befehlsformat: Codierung der Befehle

Ein Befehl besteht grundsätzlich immer aus
- der Spezifikation der Operation (Opcode)
- der Spezifikation der Operanden

Zwei mögliche Varianten:
- Einheitliches Befehlsformat
  - Alle Befehle haben eine feste Länge
  - Klassische Lösung für RISC

- Variables Befehlsformat
  - Befehle können verschieden viele und verschieden-formatige Adressen enthalten
  - Typisch für CISC
  - Extremfall: DEC VAX: 1..53 Bytes pro Befehl

- In RISC-V sind (normalerweise) alle Befehle 32bit
  - Es gibt aber auch 16bit Instruktionen, später mehr dazu
  - Längere Instruktionen können definiert werden




Binär-Code

RISC-V Opcode Structure

31 27	26 25	24 20	19 15	14 12	11 7	6 0	
funct7		rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]			rs1	funct3	rd	opcode	I-type
imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode	B-type
imm[31:12]					rd	opcode	U-type
imm[20|10:1|11|19:12]					rd	opcode	J-type

Example: Addition (***add, addw*** oder ***addi***) – von oben: `addw a5,a5,a4`

0000000 |	01111	01110	000	01111	011 1011	ADDW

imm[11:0]		rs1	000	rd	0010011	ADDI
0000000	rs2	rs1	000	rd	0110011	ADD

Einführung in die Rechnerarchitektur (ERA) IN0004

Assemblerbefehle und Operanden (am Beispiel RISC-V)

Martin Schulz
schulzm@in.tum.de
Chair for Computer Architecture and Parallel Systems
https://www.ce.cit.tum.de/caps/

Robert Wille
robert.wille@tum.de
Chair for Design Automation
https://www.cda.cit.tum.de/



Befehlssatz = Assemblersprache

Kern einer jeden „Instruction Set Architecture“ (ISA)

Drei Hauptklassen von Befehlen:

- Arithmetische und logische Operationen
- Datentransfer
- Steuerung des Programmablaufs = Sprünge und Unterprogrammaufrufe

Zusätzlich:

- Systembefehle für Verwaltungsaufgaben
  - Werden vom Betriebssystem genutzt
  - Unterstützung von Multi-Core und Mehrbenutzer
- Bei einigen Architekturen: Ein-/Ausgabebefehle
  - E/A-Geräte oft auch wie (spezielle) Speicherbereiche ansprechbar
    ➔Datentransfer-Befehle verwendbar

Ganzzahlige Arithmetik

Beispiele:

- ADD, SUB, NEG
  - identische Befehle für Zahlen mit und ohne Vorzeichen (Zweierkomplement!)

- In manchen ISAs: Spezialbefehle
  - CMP (Vergleich), INC (Hochzählen), DEC (Herunterzählen), ...

- MUL, DIV
  - i.a. je eine Variante für Zahlen mit und ohne Vorzeichen
  - Ergebnis oft in zwei Registern
    - Multiplikation: doppelt großes Ergebnis
    - Division: Ergebnis und Divisionsrest
  - Längere Ausführungszeiten

Ganzzahlige Arithmetik in RISC-V

Addition: `add a, b, c` `a = b + c`

- `add:` Mnemonic legt auszuführende Operation fest
- `b, c:` Quelloperanden (source operands) auf der die Operation ausgeführt wird
- `a:` Zieloperand (destination)

Subtraktion: `sub a, b, c` `a = b - c`

- `sub:` Mnemonic legt auszuführende Operation fest
- `b, c:` Quelloperanden (source operands) auf der die Operation ausgeführt wird
- `a:` Zieloperand (destination)

Komplexere Operationen

```
a = b + c - d;
```

```
add t, b, c  # t = b + c
sub a, t, d  # a = t - d
```

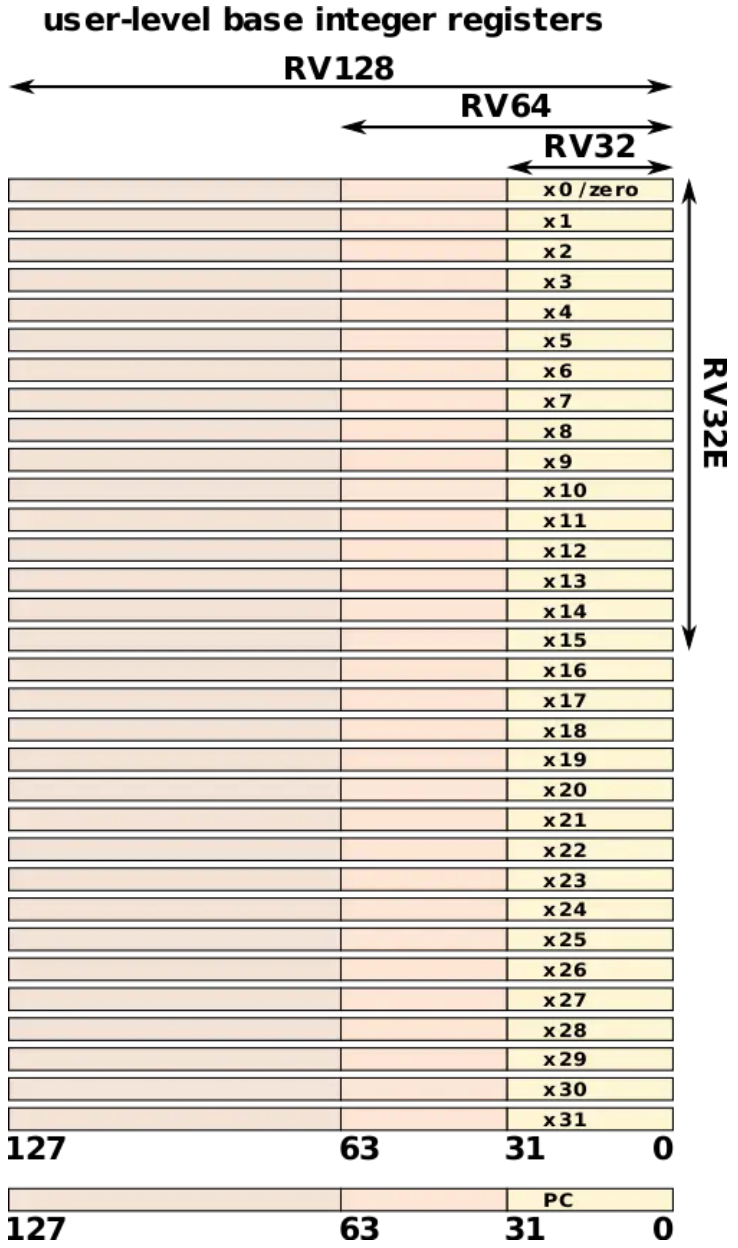
Bei RISC-V liegen alle Operanden in Registern

Register

- Kleine Anzahl von Speicherzellen
- Speichern je ein Wort in “ISA Breite”
- Innerhalb des Prozessors
- Schneller Zugriff

RISC-V ISA hat 32 bzw. 33 (sichtbare) Register

- x0 is always zero
- x1 – x31
- Spezialregister: PC (später mehr dazu)



From: https://en.wikichip.org/wiki/risc-v/registers

Bei RISC-V liegen alle Operanden in Registern

Register
- Kleine Anzahl von Speicherzellen
- Speichern je ein Wort in “ISA Breite”
- Innerhalb des Prozessors
- Schneller Zugriff

RISC-V ISA hat 32 bzw. 33 (sichtbare) Register
- x0 is always zero
- x1 – x31
- Spezialregister: PC (später mehr dazu)

Zugewiesene Aufgaben
- Konvention, aber sollte eingehalten warden
- Befehlssatz oft dafür optimiert
- Teil der ISA

Register	ABI Name	Description
x0	zero	Hard-wired zero
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5	t0	Temporary/alternate link register
x6–7	t1–2	Temporaries
x8	s0/fp	Saved register/frame pointer
x9	s1	Saved register
x10–11	a0–1	Function arguments/return values
x12–17	a2–7	Function arguments
x18–27	s2–11	Saved registers
x28–31	t3–6	Temporaries
f0–7	ft0–7	FP temporaries
f8–9	fs0–1	FP saved registers
f10–11	fa0–1	FP arguments/return values
f12–17	fa2–7	FP arguments
f18–27	fs2–11	FP saved registers
f28–31	ft8–11	FP temporaries

From: https://riscv.org/technical/specifications/

RISC-V Opcode Structure

31 27	26 25	24 20	19 15	14 12	11 7	6 0	
funct7		rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]			rs1	funct3	rd	opcode	I-type
imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode	B-type
imm[31:12]					rd	opcode	U-type
imm[20|10:1|11|19:12]					rd	opcode	J-type

Example: Addition (***add, addw*** oder ***addi***) – von oben: `addw a5,a5,a4`

0000 000	01111=15	01110=14	000	01111=15	011 1011	ADDW

imm[11:0]		rs1	000	rd	0010011	ADDI
0000000	rs2	rs1	000	rd	0110011	ADD

```
69c:   00f707bb          addw a5,a4,a5          -> R-type / 32 bit operation

       0000 0000 1111 0111 0000 0111 1011 1011
       0000 000 01111 01110 000 01111 0111011
```

Ausnahme: Befehle mit Konstanten

Konstanten können im Befehl definiert werden

- Immediate Operand
- Bei RISC-V getrennte Befehle

Addition: **`addi a, b, <imm>`** **`a = b + <imm>`**

- `addi:` Mnemonic legt auszuführende Operation fest
- `b:` Quelloperanden (source operands) auf der die Operation ausgeführt wird
- `imm:` Immediate/Konstante
- `a:` Zieloperand (destination)

Eigenschaften

- Gibt es nur für bestimmte Befehle
  - Z.B. kein „**`subi`**“ Befehl
  - Nutzung negativer Konstanten („sign extension“ wird angewendet)
- Konstanten auf 12bit beschränkt
  - Begrenzte Länge einer Instruktion

Erzeugung von 12-BIT Konstanten

Konstante (immediate) ist vorzeichenbehaftete 12-Bit-Zahl im **Zweierkomplement** (-2048 … +2047)

Beispiel: `addi`

C Code

```
// int is a 32-bit signed word
int a = -372;
int b = a + 6;
```

RISC-V Assembler

```
# Zuordnung: s0 = a, s1 = b
addi s0, zero, -372
addi s1, s0, 6
```

Alle immediates, die mehr als 12 Bits benötigen, können diese Methode nicht verwenden

Erzeugung von 32-BIT Konstanten

Größere Konstanten müssen in zwei Schritten erzeugt werden

- load upper immediate (**`lui`**) und **`addi`**
- **`lui`**: lädt eine Konstante in die oberen 20 Bits des Zielregisters und 0 in die unteren 12 Bits
- **`addi`**: addiert die unteren 12 Bits dazu

C Code

```
int a = 0xFEDC8765;
```

RISC-V Assembler

```
# s0 = a
lui  s0, 0xFEDC8    # s0 = 0xFEDC8000
addi s0, s0, 0x765
```

Wichtig:

Bei **`addi`** 12-Bit-Immediate Vorzeichenerweiterung (sign extension) verwenden

- Zusätzliche Bits mit Vorzeichenbit füllen
- Ersetzt die unteren 12 Bits und die müssen zu den oberen 20 Bits passen

Multiplikation

Nur verfügbar mit M-Erweiterung

- Multiplikation von 32-bit Zahlen kann zu 64-bit Resultat führen
- Extra Instruktionen für die oberen 32-Bit (signed und unsigned Kombinationen)

`mul rd, rs1, rs2`	Multiply	$rd := (rs1 * rs2)_{31:0}$
`mulh rd, rs1, rs2`	Multiply high signed * signed	$rd := (rs1 * rs2)_{63:32}$
`mulhsu rd, rs1, rs2`	Multiply high signed * unsigned	$rd := (rs1 * rs2)_{63:32}$
`mulhu rd, rs1, rs2`	Multiply high unsigned * unsigned	$rd := (rs1 * rs2)_{63:32}$

Beispiel:

```
s1 = 0x40000000 = 2^30
s2 = 0x80000000 = -2^31
s1 * s2 = -2^61 = 0xE0000000 00000000
s4 = 0xE0000000; s3 = 0x00000000
```

RISC-V Assembler

```
mul s3, s1, s2
mulh s4, s1, s2
# {s4,s3} = s1 * s2
```

Division und REST

Nur verfügbar mit der M-Erweiterung

- Ermitteln des Divisionsergebnisses

`div rd, rs1, rs2`	divide signed	`rd := rs1 / rs2`
`divu rd, rs1, rs2`	divide unsigned	`rd := rs1 / rs2`

- REM zum ermitteln des Rests der Division

`rem rd, rs1, rs2`	remainder signed	`rd := rs1 % rs2`
`remu rd, rs1, rs2`	remainder unsigned	`rd := rs1 % rs2`

Logische Operationen

UND: `and a, b, c` `a = b AND c`

- `and:` Mnemonic legt auszuführende Operation fest
- `b, c:` Quelloperanden (source operands) auf der die Operation ausgeführt wird
- `a:` Zieloperand (destination) → Bitweises „AND“ für jedes bit

- Ähnlich für andere Operatoren: OR, XOR
- Optionen mit Konstanten für AND, OR und XOR

AND Truth Table

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR Truth Table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

XOR Truth Table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

From: https://www.realdigital.org/doc/e127ebfa82dbc904b5c0dac5d1adce8e

Logische Befehle: Beispiel (Masken)

	Source Registers			
s1	0100 0110	1010 0001	1111 0001	1011 0111
s2	1111 1111	1111 1111	0000 0000	0000 0000

Assembly Code		Result			
`and s3, s1, s2`	s3	0100 0110	1010 0001	0000 0000	0000 0000
`or  s4, s1, s2`	s4	1111 1111	1111 1111	1111 0001	1011 0111
`xor s5, s1, s2`	s5	1011 1001	0101 1110	1111 0001	1011 0111

Schiebebefehle (Shifts)

`uimm` = Vorzeichenlose 5-Bit Zahl
`>>` = logischer Shift (Null Bits nachgeführt)
`>>>` = arithmetischen Shift (erhält Vorzeichen, 0 oder 1 nachgeführt)

Beispiel: $1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110\ 0111_{2K} = -25$

```
sra t0, t0, 4 ⇨
```

$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{2K} = -2$

`sll rd, rs1, rs2`	shift left logical	`rd := rs1 << rs2`$_{4:0}$
`slli rd, rs1, uimm`	shift left logical immediate	`rd := rs1 << uimm`
`srl rd, rs1, rs2`	shift right logical	`rd := rs1 >> rs2`$_{4:0}$
`srli rd, rs1, uimm`	shift right logical immediate	`rd := rs1 >> uimm`
`sra rd, rs1, rs2`	shift right arithmetic	`rd := rs1 >>> rs2`$_{4:0}$
`srai rd, rs1, uimm`	shift right arithmetic immediate	`rd := rs1 >>> uimm`

Weitere Informationen:
https://riscv.org/news/2020/08/logical-arithmetic-shifting-of-binary-numbers-john-johns-basement-youtube/

Gleitkomma-Arithmetik

Nicht im Basisbefehlssatz bei RISC-V enthalten

- „F“ Extension für Single Precision (32bit) Gleitkomma
- „D“ Extension für Double Precision (64bit) Gleitkomma

Eigene Register: 32bit oder 64bit Register je nach Extension

Addition: `fadd a, b, c` `a = b + c`

- `fadd:` Mnemonic legt auszuführende Operation fest
- `b, c:` Quelloperanden (source operands) auf der die Operation ausgeführt wird
- `a:` Zieloperand (destination)

Subtraktion: `fsub a, b, c` `a = b - c`

- `fsub:` Mnemonic legt auszuführende Operation fest
- `b, c:` Quelloperanden (source operands) auf der die Operation ausgeführt wird
- `a:` Zieloperand (destination)

Alle Operanden liegen in den extra 32 Gleitkomma-Registern

https://five-embeddev.com/riscv-isa-manual/latest/f.html#sec:single-float

https://five-embeddev.com/riscv-isa-manual/latest/d.html#d-standard-extension-for-double-precision-floating-point-version-2.2

Einführung in die Rechnerarchitektur (ERA) IN0004

Datentransferbehle

(Haupt) Speicher

Martin Schulz
schulzm@in.tum.de
Chair for Computer Architecture and Parallel Systems
https://www.ce.cit.tum.de/caps/

Robert Wille
robert.wille@tum.de
Chair for Design Automation
https://www.cda.cit.tum.de/



Datenspeicherung

Operanden nur aus Registern und Konstanten

- Register sind schnell, aber auch sehr begrenzt
- Nur als Zwischenspeicher gedacht

Zusätzlich: Hauptspeicher

- Große Speicherkapazität
- Ablage aller zu berarbeitenden Daten
- Nicht zum Verwechseln mit „Hintergrundspeicher“
  - Hauptspeicher meist flüchtig
  - Daten müssen in den Hauptspeicher geladen werden
- Meist als “Random Access Memory“ oder RAM
  - Man kann auf jedes gespeicherte Element zugreifen
  - Kein „vorspulen“ / „zurückspulen“

Konsequenz

- Daten müssen übertragen werden
- „Load“ und „Store“ Befehle → RISC-V ist eine „Load and Store Architecture“
- Andere ISAs erlauben Operanden direkt aus dem Hauptspeicher

Hauptspeicher Konzept

Hauptspeicher

Adresse	Wert
000	
001	
010	
011	
100	
101	
110	
111	

Menge von Speicherzellen
- Logisch üblicherweise passend zur ISA Breite
- Jede Zelle speichert einen Wert

Jede Zelle hat eine Adresse
- Üblicherweise ab Adresse 0

Feste Adresslänge
- Im Beispiel: 3bit
- Legt maximale Speichergröße fest
- Typischerweise 8bit = 1Byte Zellen können adressiert werden
- Zugriffe aber in Wortbreite des Prozessors

„Random Access Memory“ (RAM)
- Immer Zugriff auf jede Zelle möglich

Speichergrößen

Dezimal		
Wert	Abk.	Metrik
1	B	byte
$1000=10^3$	KB	kilobyte
1000^2	MB	megabyte
1000^3	GB	gigabyte
1000^4	TB	terabyte
1000^5	PB	petabyte
1000^6	EB	exabyte
1000^7	ZB	zettabyte

Speichergrößen

Dezimal			Binär		
Wert	Abk.	Metrik	Wert	Abk.	Metrik
1	B	byte	1	B	byte
$1000=10^3$	KB	kilobyte	$1024 = 2^{10}$	KiB	kibibyte
1000^2	MB	megabyte	1024^2	MiB	mebibyte
1000^3	GB	gigabyte	1024^3	GiB	gibibyte
1000^4	TB	terabyte	1024^4	TiB	tebibyte
1000^5	PB	petabyte	1024^5	PiB	pebibyte
1000^6	EB	exabyte	1024^6	EiB	exbibyte
1000^7	ZB	zettabyte	1024^7	ZiB	zebibyte

Ein Adressraum mit folgender Adresslänge hat folgende Größe

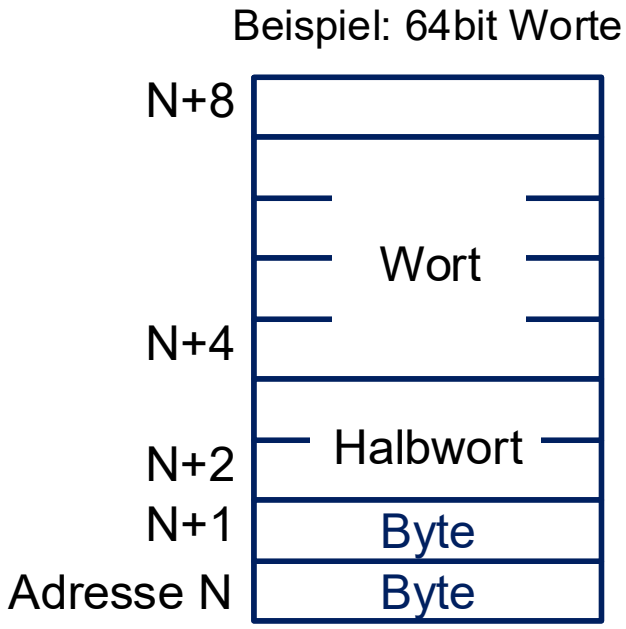
- 16 bit: 2^{16} = 64 KiB
- 32 bit: 2^{32} = 4 GiB
- 64 bit: 2^{64} = 16 EiB

Speicher/Zellenbreite

Speicherzellen auf ISA Ebene sind üblicherweise mehrere Byte breit

Entspricht der Verarbeitungsgröße
- 16bit vs. 32bit vs. 64bit vs, 128bit Rechner
- Anzahl der Datenleitungen
- Warum? Effizienterer Transfer, Einfachere Logik

In der Praxis sind Speicher byte-adressierbar
- Adresse eines Datums bezieht sich immer auf Bytegrenzen
- Unabhängig vom Datenformat
- Mehrere Adressen zeigen in eine Zelle



Aber: Unterschiede bei der Behandlung größerer Datenformate
- Ausrichtung der Daten im Speicher
- Byte-Reihenfolge

Ausrichtung der Daten im Speicher

Definition: Ausrichtung von Daten („data alignment“)

- Ein Datum ist auf eine n-Byte-Grenze ausgerichtet,
  wenn seine Adresse A ein ganzzahliges Vielfaches von n ist:
  A mod n = 0
- Bei Ausrichtung auf „Datenbreite“ der ISA → immer nur ganze „Wort-Zellen“

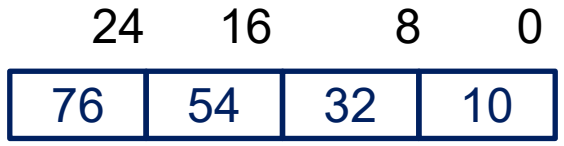
Häufige Forderung

- Eine solche Ausrichtung ist oft gewünscht oder gefordert
  - Gewünscht: schnellere Zugriffe, aber nicht ausgerichtete Zugriffe funktionieren (x86)
  - Gefordert: nicht ausgerichtete Zugriffe führen zu einem Fehler (ARM bei manchen Typen)
  - Option: getrennte Befehle für ausgerichtete und nicht ausgerichtete Zugriffe (MIPS, RISC-V)

- Hintergrund: Speicher auf ISA Ebene ist in Worten organisiert
  - Je nach Datenbusbreite liefert ein Zugriff z.B. 32 Bit oder 64 Bit
  - Bei nicht ausgerichteten Daten: Zugriff über Zellgrenzen hinweg
    - Mehrere Zugriffe
  - Bei ausgerichteten Operanden: minimale Anzahl von Speicherzugriffen

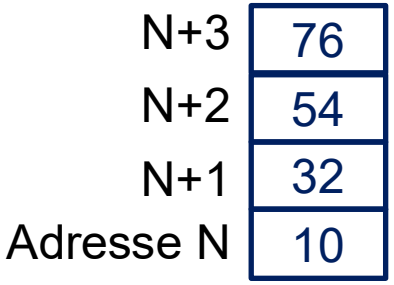
Byte-Reihenfolge

Beispiel: Ablage des Worts im Speicher



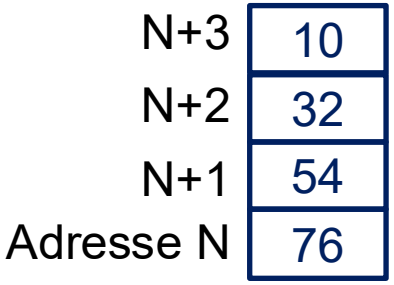
=0x76543210

Little Endian



Beispiel: Intel IA-32, typisch für PC

Big Endian



Beispiel: SPARC, Motorola, MIPS64

- Heute: Little Endian dominiert (x86 / x86_64)
- Die meisten anderen Architekturen bieten Wechselmöglichkeit (auch RISC-V)

RISC-V Befehle zum Lesen vom Speicher

Lesen in Speicher: **`ld (load doubleword)`** **`64bit`**

Format:

```
ld destination, offset(base)
```

Beispiel:

```
ld a1, 8(s0)
```

Adressberechnung:

- Basisadresse (`s0`) zum *Offset* (8) addieren
- Adresse = ([Wert in `s0`] + 8)

Ergebnis:

- Der Wert in der Hauptspeicherzelle mit der Adresse **`offset+base`** (im Beispiel **`s0+8`**) wird in das Register **`destination`** geladen
- Bemerkung: Jedes Register kann als Basisadresse (**`base`**) verwendet werden

RISC-V Befehle zum Schreiben in den Speicher

Lesen in Speicher: **`sd (store doubleword)`** **`64bit`**

Format:

```
sd source, offset(base)
```

Beispiel:

```
sd a1, 8(s0)
```

Adressberechnung:

- Basisadresse (`s0`) zum *Offset* (8) addieren
- Adresse = ([Wert in `s0`] + 8)

Ergebnis:

- Der Wert im Register **`source`** wird in die Hauptspeicherzelle mit der Adresse **`offset+base`** (im Beispiel **`s0+8`**) geschrieben
- Bemerkung: Jedes Register kann als Basisadresse (**`base`**) verwendet werden

Einführung in die Rechnerarchitektur (ERA) IN0004

Sprünge

Martin Schulz
schulzm@in.tum.de
Chair for Computer Architecture and Parallel Systems
https://www.ce.cit.tum.de/caps/

Robert Wille
robert.wille@tum.de
Chair for Design Automation
https://www.cda.cit.tum.de/



Steuerung des Programmablaufs: Sprungbefehle

Unbedingter Sprung

- Programm wird an anderer Stelle fortgesetzt
- Implementierung: Neusetzen des PC

`j imm`: Jump: Programm wird an PC+imm fortgesetzt (kurzer relativer Sprung)

`jr reg,imm`: Jump Register: Programm wird an Register Wert + imm fortgesetzt

Unbedingter Sprung

```
    j           target          # Springe an Adresse mit label "target"
    srai        s1, s1, 2       # nicht ausgeführt
    addi        s1, s1, 1       # nicht ausgeführt
    sub         s1, s1, s0      # nicht ausgeführt

target:
    add         s1, s1, s0      # s1 = 1 + 4 = 5
```

In Assemblersprache können Sprungmarken verwendet werden

- Im Beispiel „`target`“
- Label dürfen keine reservierten Bezeichner sein
- Definition mit Doppelpunkt abgeschlossen
- Wird im Assembler/letzten Compilerschritt durch passenden „Offset“ ersetzt

Steuerung des Programmablaufs: Sprungbefehle

Unbedingter Sprung

- Programm wird an anderer Stelle fortgesetzt
- Implementierung: Neusetzen des PC

`j imm`: Jump: Programm wird an PC+imm fortgesetzt (kurzer relativer Sprung)

`jr reg,imm`: Jump Register: Programm wird an Register Wert + imm fortgesetzt

Bedingter Sprung

- Sprung nur wenn eine Bedingung erfüllt ist
- Bedingung muss kodiert werden
- Notwendig für Schleifen (Sprung zurück wenn Schleifenende noch nicht erreicht)

Format: `bxx r1,r2,imm`

Falls Bedingung xx erfüllt ist, wird das Programm an PC+imm fortgeführt
Ansonsten beim nächsten Befehl

Bedingungen:	branch if equal (`beq`)	$r1 = r2$
	branch if not equal (`bne`)	$r1 \neq r2$
	branch if less than (`blt`)	$r1 < r2$
	branch if greater than or equal (`bge`)	$r1 \geq r2$

Bedingter Sprung

Option 1

```
 addi s0, zero, 4          # Add Immediate / s0 = 0 + 4 = 4
 addi s1, zero, 1          # Add Immediate / s1 = 0 + 1 = 1
 slli s1, s1, 2            # Shift Left / s1 = 1 << 2 = 4
 beq  s0, s1, target       # s0 = s1: Sprung wird ausgeführt (eq)
 addi s1, s1, 1            # nicht ausgeführt
 sub  s1, s1, s0           # nicht ausgeführt
```

Option 2

```
 addi s0, zero, 4          # Add Immediate / s0 = 0 + 4 = 4
 addi s1, zero, 1          # Add Immediate / s1 = 0 + 1 = 1
 slli s1, s1, 2            # Shift Left / s1 = 1 << 2 = 4
 bne  s0, s1, target       # s0 = s1: Sprung nicht wird ausgeführt (ne)
 addi s1, s1, 1            # Addition: s1 = 4 + 1 = 5
 sub  s1, s1, s0           # Subtraktion: s1 = 5 - 4 = 1
```

Sprungziel

```
target:                    # label
 add  s1, s1, s0           # s1 = 4 + 4 = 8 (beq)      s1 = 1 + 4 = 5 (bne)
```

Implementierung einer IF Anweisung

Assembler überprüft die negierte Bedingung `(i != j)` des Codes der höheren Programmiersprache `(i == j)` um die „if“ Anweisungen zu überspringen

C Code

```
if (i == j)
  f = g + h;

f = f - i;
```

RISC-V Assembler

```
# s0 = f, s1 = g, s2 = h
# s3 = i, s4 = j
    bne s3, s4, L1
    add s0, s1, s2
L1:
    sub s0, s0, s3
```

Implementierung einer IF/ELSE Anweisung

Assembler überprüft die negierte Bedingung `(i != j)` des Codes der höheren Programmiersprache `(i == j)` um die „if“ Anweisungen zu überspringen

C Code

```
if (i == j)
  f = g + h;

f = f - i;
```

RISC-V Assembler

```
# s0 = f, s1 = g, s2 = h
# s3 = i, s4 = j
    bne s3, s4, L1
    add s0, s1, s2
L1:
    sub s0, s0, s3
```

```
if (i == j)
  f = g + h;


else
  f = f - i;
```

```
    bne s3, s4, L1
    add s0, s1, s2
    j   done

L1:
    sub s0, s0, s3
done:
```

Implementierung einer While Schleife

Assembler überprüft die negierte Bedingung `(pow == 128)` des Codes der höheren Programmiersprache `(pow != 128)` um Ende der Schleife zu testen

C Code

```
// Berechne x, so dass
// 2^x = 128

int pow = 1;
int x   = 0;

while (pow != 128) {
  pow = pow * 2;
  x = x + 1;
}
```

RISC-V Assembler

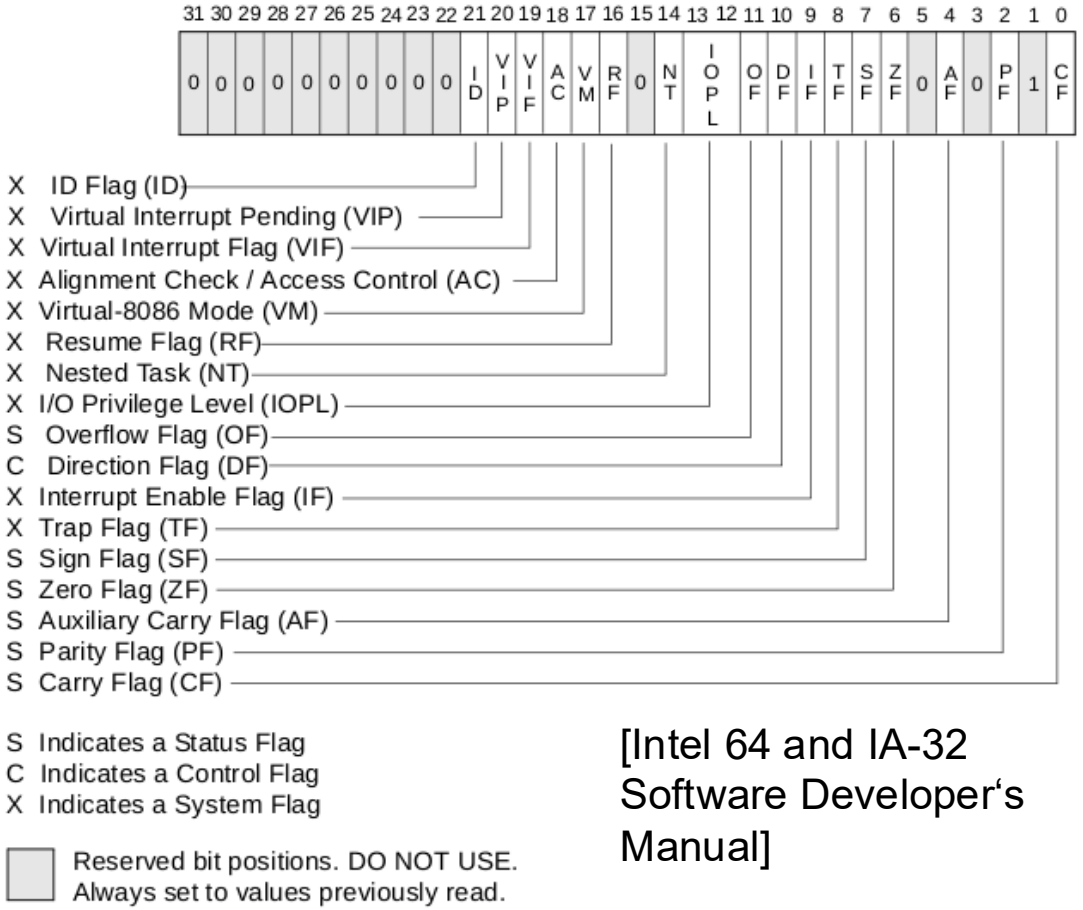
```
# s0 = pow, s1 = x


        addi s0, zero, 1
        add  s1, zero, zero
        addi t0, zero, 128
while:
        beq  s0, t0, done
        slli s0, s0, 1
        addi s1, s1, 1
        j    while
done:
```

Alternative (z.B. x86): Statusregister

Extra Register in der ISA um Bedingungen zu speichern

- Basierend auf letzter arithmetischen Operation



[Intel 64 and IA-32 Software Developer's Manual]

JZ — Jump if Zero; checks for ZF = 1
JE — Jump if Equal; checks for ZF = 1
JNZ — Jump if Not Zero; checks for ZF = 0
JNE — Jump if Not Equal; checks for ZF = 0
JC — Jump if Carry; checks for CF = 1
JNC — Jump if No Carry; checks for CF = 0
JS — Jump if Signed (Negative); checks for SF = 1
JG — Jump if Greater
JNLE — Jump if Not Less or Equal
JGE — Jump if Greater or Equal
JNL — Jump if Not Less
JL — Jump if Less
JLE — Jump if Less or Equal
JNG — Jump if Not Greater

From: https://resources.infosecinstitute.com/topics/secure-coding/conditionals-and-jump-instructions/

Einführung in die Rechnerarchitektur (ERA) IN0004

Unterprogramme

Martin Schulz
schulzm@in.tum.de
Chair for Computer Architecture and Parallel Systems
https://www.ce.cit.tum.de/caps/

Robert Wille
robert.wille@tum.de
Chair for Design Automation
https://www.cda.cit.tum.de/



Steuerung des Programmablaufs: Unterprogrammaufruf

Bisher

- Sequentielle Programmabfolge
- Sprünge sind möglich → Schleifen
- Monolithischer Block

Was fehlt noch?

- Aufruf von gemeinsamer Funktionalität
  - Strukturierte Programmierung
  - Bibliotheken/Libraries von anderen ProgrammiererInnen

Unterprogrammaufruf

- Sprung an eine Routine
- Vor dem Sprung: Sichern der Rückkehradresse
- Am Ende der Routine, Rücksprung an diese Adresse
- Konventionen um sicher zu stellen, dass nichts überschrieben wird
  - Welche Information muss gesichert werden, welche kann überschrieben werden?
  - Beliebige Routinen müssen zusammenpassen können

Unterprogramme in RISC-V

Begriffe

Caller = Aufrufende Funktion (im Beispiel `main`)

Callee = Aufgerufene Funktion (im Beispiel `simple`)

`jal offset:` `ra (x1)` = PC + 4 (0x00000304) bzw. nächster Befehl

Sprung zu PC+offset (im Beispiel label **`simple`** bei 0x0000051c)

Rücksprung: **`jr ra`** PC = `ra` (0x00000304)

C Code

```
int main() {
  simple();
  a = b + c;
}

void simple() {
  return;
}
```

RISC-V Assembler

```
0x00000300 main:   jal  simple      # call
0x00000304         add  s0, s1, s2
...                ...

0x0000051c simple: jr   ra          # return
```

Argumente und Rückgabewert (1)

C Code

```
int main()
{
  int y;
  ...
  y = diffofsums(2, 3, 4, 5);  // 4 arguments
  ...
}

int diffofsums(int f, int g, int h, int i)
{
  int result;
  result = (f + g) - (h + i);
  return result;                // return value
}
```

Aufrufkonvention

Soviel wie möglich via Register

- Eingabedaten
- Rückgabewert

Gezielte Aufgaben

- a0-a7: Eingaben
- a0-a1: Ergebnisse
- ra: Rücksprungadresse

Caller-saved

- Aufrufer/Caller muss Werte selbst sichern, dürfen von Callee verändert werden

Callee-saved

- Aufgerufene Funktion darf Werte nicht verändern oder muss sie wieder herstellen

Register	ABI Name	Description	Saver
x0	zero	Hard-wired zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5–7	t0–2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10–11	a0–1	Function arguments/return values	Caller
x12–17	a2–7	Function arguments	Caller
x18–27	s2–11	Saved registers	Callee
x28–31	t3–6	Temporaries	Caller
f0–7	ft0–7	FP temporaries	Caller
f8–9	fs0–1	FP saved registers	Callee
f10–11	fa0–1	FP arguments/return values	Caller
f12–17	fa2–7	FP arguments	Caller
f18–27	fs2–11	FP saved registers	Callee
f28–31	ft8–11	FP temporaries	Caller

From: https://riscv.org/technical/specifications/

Argumente und Rückgabewert (2)

RISC-V Assembler

```
main:
addi a0, zero, 2   # Argument 0 = 2
addi a1, zero, 3   # Argument 1 = 3
addi a2, zero, 4   # Argument 2 = 4
addi a3, zero, 5   # Argument 3 = 5
jal  diffofsums    # Aufruf
add  s7, a0, zero # Variable y in s7
```

```
...
y = diffofsums(2, 3, 4, 5);
...
int diffofsums(int f, int g, int h, int i)
{
  int result;
  result = (f + g) - (h + i);
  return result;
}
```

```
diffofsums:
add  t0, a0, a1    # t0 = f + g
add  t1, a2, a3    # t1 = h + i
sub  t2, t0, t1    # result = (f + g) - (h + i)
add  a0, t2, zero # Ergebnis in a0 speichern
jr   ra            # Rücksprung
```

Register t0,t1,t2 werden überschrieben!

Bedarf an weiteren Speichermöglichkeiten

Anzahl der Register ist immer limitiert

- Begrenzte Anzahl von Parametern
- Begrenzte Anzahl von temporären Werten
- Keine Möglichkeit Werte in temporären Registern zu sichern
- Aufruf von Unterprogrammen in Unterprogrammen
- Rekursion

Nötig Hauptspeicher zu verwenden

- Feste Speicherbereiche passen hier aber nicht
- Dynamischer Aufruf von ein oder mehreren Unterprogrammen

Temporäre Speicherbereiche

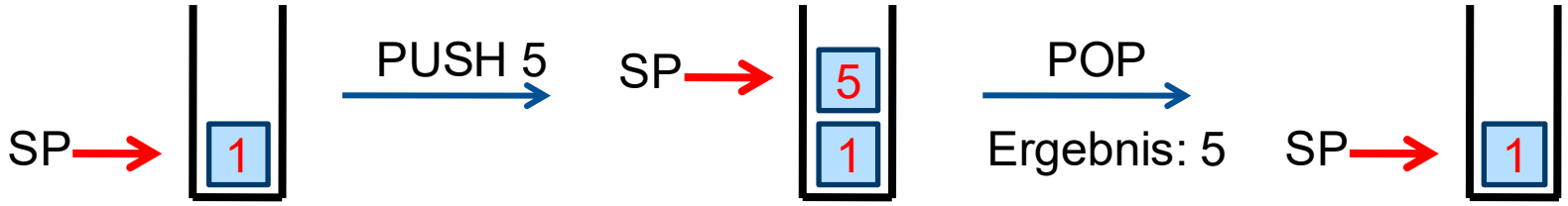
- Angelegt beim Unterprogrammaufruf
- Verfügbar um lokale Daten zu speichern
  - Weitere Parameter und temporäre Daten
  - Rücksprungadresse für diesen (einen) Aufruf
- Freigabe bei Rücksprung

Passende Datenstruktur: Keller/Stapel/Stack

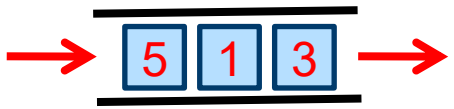
Liste von Zahlen, der man auf einer Seite

- Werte hinzufügen kann (PUSH)
- Werte entnehmen kann (POP)
- Meist übereinander gezeichnet
- Das Ende wird markiert durch eine Zeiger (Stack Pointer)

LIFO Prinzip: Last In First Out



Gegenteil:
FIFO Prinzip: First In First Out
Warteschlange/Queue

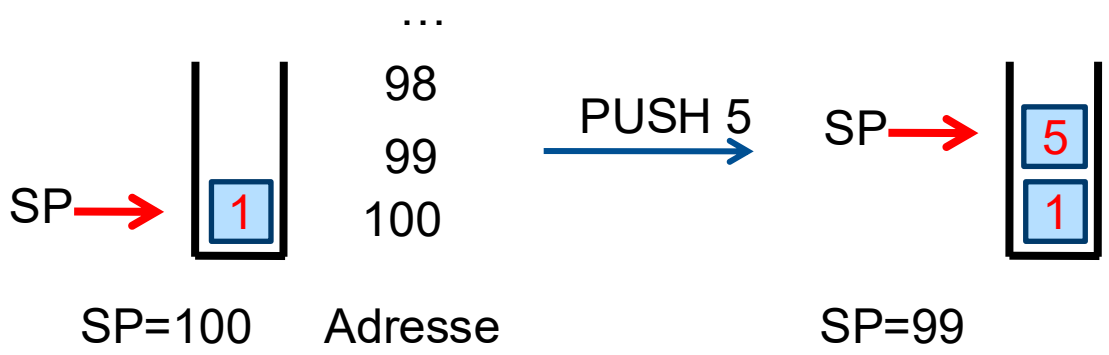


Implementierung von Kellern/Stacks

Keller für Programmausführung werden oft im Speicher so abgelegt, dass sie „nach unten“ wachsen: Richtung kleinere Addressen

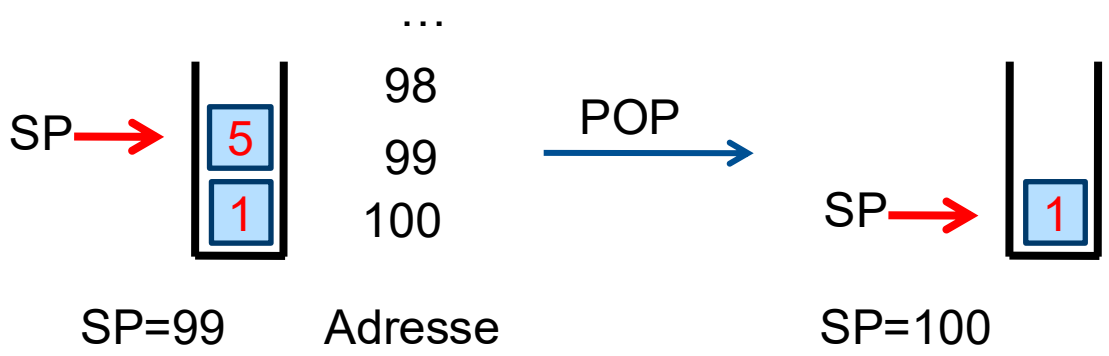
PUSH Funktionalität

- SP erniedrigen und in Adresse SP schreiben
- Prädekrement
- Viele ISAs haben dafür spezielle Befehle
- RISC-V
  - Anpassung von SP mit arithmetischen Befehlen
  - Schreiben mit „sd“ Anweisungen



POP Funktionalität

- Von Adresse SP lesen und SP erhöhen
- Postinkrement
- Viele ISAs haben dafür spezielle Befehle
- RISC-V
  - Lesen mit „ld“ Anweisungen
  - Anpassung von SP mit arithmetischen Befehlen



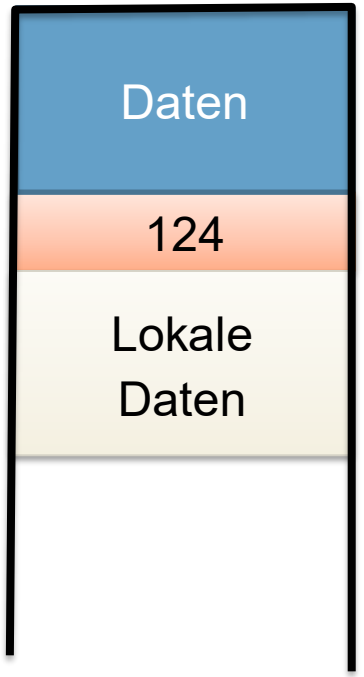
Unterprogrammaufruf mit Kellern/Stacks

Unterprogramm

- Implementiert bestimmte Funktionalität
- Wird bei Bedarf aufgerufen
- Nach Ende, weiter Ausführung von der Stelle des Aufrufes

```
100: Work
…
123: Call compute (200)
124: Work
…

Compute:  200: get data
          201: do arithm
          …
          212: store data
          213: return
```



Genaue Details durch
“Aufrufkonvention”
definiert, als Teil der ISA

Typische Speicherverwaltung

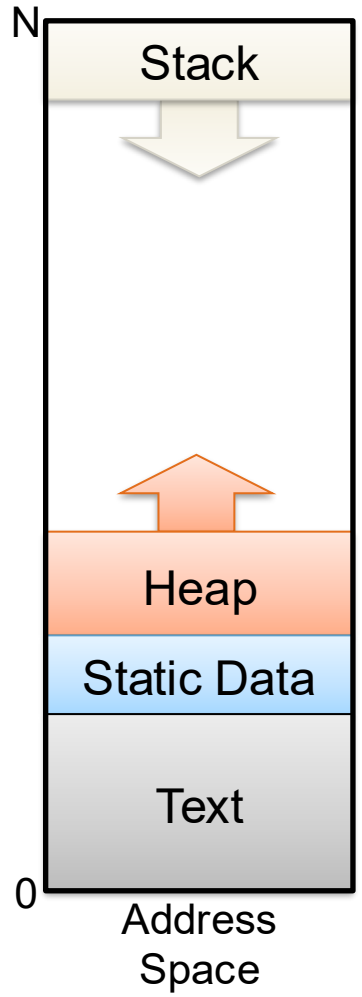
Jedes Programm hat einen Adressraum
- Enthält Daten und Programm

Code (”text”) wird meistens am unteren Ende geladen

Statische Daten
- Fest allozierte Daten als Teil des Codes

Dynamische Speicherverwaltung
- Auch “Heap” genannt
- Kann angefragt oder freigegeben werden
- Wächst nach oben

Keller/Stack wächst nach unten
- Optimale Platzausnutzung
- Wenn sie sich treffen -> Fehler



Einführung in die Rechnerarchitektur (ERA) IN0004

Sondervarianten in RISC-V

Martin Schulz
schulzm@in.tum.de
Chair for Computer Architecture and Parallel Systems
https://www.ce.cit.tum.de/caps/

Robert Wille
robert.wille@tum.de
Chair for Design Automation
https://www.cda.cit.tum.de/



Steuerung des Programmablaufs: Sprungbefehle

Unbedingter Sprung = Programm wird an anderer Stelle fortgesetzt

- Sprung: Neusetzen des PC ohne Nebenwirkung
  - `j imm`: Jump: Programm wird an PC+imm fortgesetzt (kurzer relativer Sprung)
  - `jr reg,imm`: Jump Register: Programm wird an Register Wert + imm fortgesetzt

- Unterprogrammaufruf: Neusetzen des PC plus Merken des PCs (für Rücksprung)
  - `jal imm`: Jump zu PC+imm, Rücksprungadresse in `ra` gespeichert
  - `jalr reg,imm`: Jump zu Register Wert + imm, Rücksprungadresse in `ra` gespeichert

Pseudobefehle und Abkürzungen

Unbedingter Sprung = Programm wird an anderer Stelle fortgesetzt

- Sprung: Neusetzen des PC ohne Nebenwirkung

```
j imm          = jal x0,offset              (jump and link)
jr reg,imm     = jalr x0,reg,offset         (jump and link register)
```

- Unterprogrammaufruf: Neusetzen des PC plus Merken des PCs (für Rücksprung)

```
jal imm        = jal x1,offset              ra = x1
jalr reg,imm   = jalr x1,reg,offset         ra = x1
```

- Weiterer Pseudo-Befehl

```
mv ra,rb       =      add ra,rb,x0          oder   addi ra,rb,0
```

Andere ISAs haben hier Spezialbefehle

- Call Befehle
- Return Befehle
- Implizite Speicherung der Rücksprungadresse (bei RISC-V ist das explizit)

Bedingte Sprünge

Bedingter Sprung = Sprung nur wenn eine Bedingung erfüllt ist (keine Speicherung von `ra`)

```
bxx r1,r2,imm
```

Bedingungen:	branch if equal (`beq`)	$r1 = r2$
	branch if not equal (`bne`)	$r1 \neq r2$
	branch if less than (`blt`)	$r1 < r2$
	branch if greater than or equal (`bge`)	$r1 \geq r2$

`beqz rs, offset`	`beq rs, x0, offset`	Branch if $=$ zero
`bnez rs, offset`	`bne rs, x0, offset`	Branch if $\neq$ zero
`blez rs, offset`	`bge x0, rs, offset`	Branch if $\leq$ zero
`bgez rs, offset`	`bge rs, x0, offset`	Branch if $\geq$ zero
`bltz rs, offset`	`blt rs, x0, offset`	Branch if $<$ zero
`bgtz rs, offset`	`blt x0, rs, offset`	Branch if $>$ zero
`bgt rs, rt, offset`	`blt rt, rs, offset`	Branch if $>$
`ble rs, rt, offset`	`bge rt, rs, offset`	Branch if $\leq$
`bgtu rs, rt, offset`	`bltu rt, rs, offset`	Branch if $>$, unsigned
`bleu rs, rt, offset`	`bgeu rt, rs, offset`	Branch if $\leq$, unsigned